ARCHITECTURE OVERVIEW

Spatial Blocks

RTX Edition / Queries, scene updates, rendering

ovspatialquery
Validate placement

ovstage
Publish scene state

ovrtx
Render the 3D view

Spatial Blocks demonstrates how spatial queries and RTX rendering can work together in a compact interactive application. It combines classic 10 × 20 falling-block gameplay with a fully 3D scene, textured beveled pieces, a landing preview, reflections, and animated effects.

ovspatialquery answers "Can this piece fit?" Python owns the game rules, including gravity, rotation, scoring, and row clearing. Settled blocks are instances of a shared box geometry in an explicit spatial population. Candidate movements, rotations, and landing positions use the real native CPU **overlap_any** API with exact oriented-box queries. Slightly inset query boxes allow neighboring blocks to touch. Settling and clearing pieces publish changed and removed objects while preserving the identities of surviving blocks.

ovstage manages the visible world. An OpenUSD scene defines the board, meshes, materials, lights, and camera. Preallocated objects represent pieces, landing guides, sparks, and light trails. Reusable queries update their transforms and material bindings. Each update receives a version number, called an ordinal, and is published before rendering so ovrtx consumes a coherent scene state.

ovrtx turns that state into an image. Its real-time path-tracing mode runs on the NVIDIA GPU, producing lighting, shadows, reflections, and bloom. Game events drive the 3D effects. Rendered pixels are mapped to CPU memory through DLPack, copied, and released, then encoded as JPEG frames for the local browser interface.

A single Python thread owns gameplay and native rendering; browser inputs arrive through a queue. This keeps scene updates and rendering ordered while the interface remains separate. The architecture illustrates a useful separation of responsibilities: game logic decides what should happen, spatial queries validate placement, scene management publishes changes, and RTX renders the result.

The accompanying 14-second clip uses a prepared board and scripted inputs through the real libraries, with explanatory captions. It is a functional demonstration, not a performance benchmark.